GOOGLE | 1POINT3ACRES | TOPIC 3 OF 10 | CORE TOPIC

# Graphs and Search

11 questions to study here, each on its own page, with the full interview answer and code

6 more fit this topic but are covered in another topic: they are listed as SKIP at the end of the contents

| 1 | 10 |
|---|---|
| HARD | MEDIUM |

**How to use**

- Questions are Hard first, then Medium, then Easy, newest last-asked date first.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Graphs and Search | 11 questions | tap a title to jump to it**


| | | |
|---|---|---|
| **H** | 1. Friend Graph with Timeline (Friend / Unfriend Events) | **2026-09-29** |
| **M** | 2. Template Variable Expansion with Follow-Ups | **2026-10-17** |
| **M** | 3. Train / Flight Schedule Reachability | **2026-10-08** |
| **M** | 4. Course Scheduling and Dependency Variants | **2026-10-04** |
| **M** | 5. Number of Islands | **2026-09-30** |
| **M** | 6. Path Minimizing the Maximum Cell (Swim in Rising Water) | **2026-09-29** |
| **M** | 7. Shortest Path Nodes and Escaping a Pursuer | **2026-09-29** |
| **M** | 8. Rotten Oranges / Multi-Source BFS (taxis) | **2026-09-08** |
| **M** | 9. 2D Points Connected Components by Radius | **2026-08-04** |
| **M** | 10. Minimum Direction Violations in a Directed Graph | **2026-07-19** |
| **M** | 11. Shortest Directed Cycle Through a Node | |


**SKIP: these fit this topic but have their pages in another topic (6)**


| | | |
|---|---|---|
| **H** | SKIP Tree Distance Sum (Re-root DP) | **topic 2: Dynamic Programming** |
| **M** | SKIP URL Request Handler Router | **topic 5: Trees and Tries** |
| **M** | SKIP Expression Tree Single-Leaf Mutation | **topic 5: Trees and Tries** |
| **M** | SKIP Compute File System Total Size | **topic 5: Trees and Tries** |
| **M** | SKIP Longest Consecutive Path in Binary Tree | **topic 5: Trees and Tries** |
| **M** | SKIP Trie Prefix Search / Autocomplete | **topic 5: Trees and Tries** |

**STUDY HERE: topic 3, Graphs and Search**

Site tags: graph, union-find, hard

**HARD | 1Point3Acres | onsite-coding, phone-screen | 1 of 11**

# Friend Graph with Timeline (Friend / Unfriend Events)

**ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 186/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** written and tested here for this exact question

Frequency: Medium | Last asked: 2026-09-29 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/friend-graph-with-timeline

## QUESTION

**EXAMPLES**

```
Base: N=3, events [(1,A,B,friend), (5,B,C,friend)] ->
    answer 5.
Follow-up 1: after the above events, query (0, A, C)
    -> false, (5, A, C) -> true.
```

## APPROACH

Have weighted/path-compressed union-find ready and warm. Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.

Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask what the answer is for the base question (earliest time all N people are connected), what to return if it never happens, whether unfriend events exist, and whether queries ask about a past time.

**BRUTE FORCE TO SAY OUT LOUD**

For each query rebuild the graph from the events up to that time and run BFS. O(E + N) per query.

**OPTIMAL IDEA**

Base: sort events and union-find, return the time the component count hits 1. Query at a time t: union by size without path compression and record each union's time; two people are connected at t if the largest time on the path between them is at most t. Unfriend: offline, place each edge's lifetime into a segment tree over time and run DFS with a rollback union-find, checking connectivity at the leaves.

**DRY-RUN THESE CASES**

## INTERVIEW-DAY PLAN

The example giving 5. A graph that never connects. Two events at the same time. Unfriend then refriend.

**LIKELY FOLLOW-UPS**

Online unfriend (Holm-de Lichtenberg-Thorup or link-cut trees), counting components over time.

**PITFALLS**

Using path compression where rollback is needed. Treating unfriend like union-find undo.

## HOW TO THINK ABOUT IT

Everyone becomes connected at the first moment the graph has one component. If friendships only appear, sort by time and use union-find (components only merge). Unfriending breaks that: removing an edge can split a component and union-find cannot undo. With all events known up front, handle each edge's lifetime in a segment tree over time and use a rollback union-find.

**Basis:** LeetCode 1101, The Earliest Moment When Everyone Become Friends, plus connectivity queries over time and unfriend events (offline dynamic connectivity). Assumption: events are (time, a, b, 'friend'|'unfriend').

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(n, events):
    for t in sorted({e.time for e in events}):
        graph = build_graph(events, upto=t)
        if dfs_count(graph, start=0) == n:
            return t  # everyone reached
    return -1
```

| TIME | SPACE |
|---|---|
| O(T * (N + E)) for T distinct times | O(N + E) |

**WHY IT IS SLOW**

Each time point replays all events and searches the whole graph, O(T * (N + E)). A union-find over sorted events (friend-only) is near-linear; unfriend events need offline dynamic connectivity.

## OPTIMAL SOLUTION · Base: earliest time everyone is connected (Union-Find on sorted events)

| TIME | SPACE |
|---|---|
| O(E log E) | O(N) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 Why does unfriending break plain Union-Find?**

A union merges two sets and forgets which edge caused it, so removing that edge cannot tell which members should separate. Online, fully dynamic connectivity needs heavy structures (Holm-de Lichtenberg-Thorup or link-cut trees, polylogarithmic time). When all events are known in advance, the offline segment-tree-over-time plus rollback DSU is the practical answer and the one to code in an interview.

**FOLLOW-UP 2 What is the simple answer if the interviewer wants something quick?**

Rebuild the graph from the events up to time t and run BFS or DFS for each query: O(E + N) per query. State the cost and then explain how the offline method amortizes it. A correct simple answer is accepted; naming the optimal one shows depth.

**FOLLOW-UP 3 Report how many friend groups exist after each event (friends only).**

Union-find with a component counter: decrement when a union merges two different roots; record the counter after each event. O(E alpha(N)).

**FOLLOW-UP 4 Queries ask 'were A and B connected at time t?' with no unfriending.**

Union by size without path compression, recording the time of each union on the edge it creates; two people are connected at t if the largest time on the path between them is at most t.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict, deque


# HELPERS for the base question
def by_time(events):
    # events are (time, a, b, "friend" or "unfriend")
    return sorted(events, key=lambda e: e[0])


# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
# FOLLOW-UP 4: Queries ask 'were A and B connected at time
# t?' with no unfriending.
class TimedUnionFind:
    """Union by size, no path compression, so every
    link keeps the time it was made."""

    def __init__(self, n):
        # every person starts alone in a group of one
        self.parent = list(range(n))
        # sizes decide which group hangs under which
        self.size = [1] * n
        # time a node was linked under its parent
        self.link_time = [0] * n
        # how many separate groups exist right now
        self.groups = n

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, t):
        # find the leaders of both groups
        ra, rb = self.find(a), self.find(b)
        # already in one group: nothing changes
        if ra == rb:
            return False
        # make ra the bigger group
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        # hang the smaller group under the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        # remember when this link appeared
        self.link_time[rb] = t
        # two groups became one
        self.groups -= 1
        return True

    def connected_at(self, a, b, t):
        # a person is always connected to themselves
        if a == b:
            return True
        # climb from a, noting the newest link
        seen, newest, x = {a: 0}, 0, a
        while self.parent[x] != x:
            newest = max(newest, self.link_time[x])
            x = self.parent[x]
            seen[x] = newest
        # climb from b until we reach an ancestor of a
        newest, y = 0, b
        while y not in seen:
            # b's leader reached without meeting a
            if self.parent[y] == y:
                return False
            newest = max(newest, self.link_time[y])
            y = self.parent[y]
        # connected at t if all path links are <= t
        return max(newest, seen[y]) <= t


# BASE QUESTION (optimal): earliest time everyone is
# connected (Union-Find on sorted ...
def earliest_all_connected(n, events):
    # zero or one person is connected from the start
    if n <= 1:
        return 0
    uf = TimedUnionFind(n)
    # handle friendships in time order
    for t, a, b, kind in by_time(events):
        # the base question has friend events only
        if kind == "friend":
            uf.union(a, b, t)
            # one group left: everyone is connected now
            if uf.groups == 1:
                return t
    # the groups never merged into one
    return -1


# FOLLOW-UP 1, 2: Why does unfriending break plain Union-
# Find?
```

```
def connected_with_unfriends(n, events, a, b, t):
    # replay every event up to time t on a set of edges
    edges = set()
    for time, x, y, kind in by_time(events):
        if time > t:
            break
        pair = (min(x, y), max(x, y))
        # friend adds the edge, unfriend removes it
        if kind == "friend":
            edges.add(pair)
        else:
            edges.discard(pair)
    # build neighbour lists from the surviving edges
    adj = defaultdict(list)
    for x, y in edges:
        adj[x].append(y)
        adj[y].append(x)
    # breadth-first search from a looking for b
    seen, queue = {a}, deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in adj[u]:
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False


# FOLLOW-UP 3: Report how many friend groups exist after
# each event (friends only).
def groups_after_each_event(n, events):
    uf = TimedUnionFind(n)
    counts = []
    # replay friend events in time order
    for t, a, b, kind in by_time(events):
        if kind == "friend":
            uf.union(a, b, t)
            # record the group count after this event
            counts.append(uf.groups)
```

```
    return counts


if __name__ == "__main__":
    # people A, B, C are 0, 1, 2
    ev = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]
    # everyone is connected at time 5
    assert earliest_all_connected(3, ev) == 5
    # a missing friendship means never connected
    assert earliest_all_connected(3, ev[:1]) == -1
    assert earliest_all_connected(1, []) == 0
    # groups: 3 -> 2 -> 1
    assert groups_after_each_event(3, ev) == [2, 1]
    # A and C are apart at time 0, together at time 5
    uf = TimedUnionFind(3)
    for t, a, b, k in ev:
        uf.union(a, b, t)
    assert not uf.connected_at(0, 2, 4)
    assert uf.connected_at(0, 2, 5)
    # an unfriend breaks the link again
    ev2 = ev + [(7, 1, 2, "unfriend")]
    assert connected_with_unfriends(3, ev2, 0, 2, 6)
    assert not connected_with_unfriends(3, ev2, 0, 2, 8)
```

## NOTES FROM CANDIDATES

The base problem looks innocent; the follow-up about unfriending is the actual signal. In practice, interviewers accept "rebuild graph from scratch per query + BFS" as a working answer if you also articulate why the optimal algorithm is fully dynamic connectivity.

The same family shows up as "friends form a chain over time" or "chess pieces R / L that can move plus walls" — the connectivity backbone is identical.

The follow-up the interviewer is fishing for is consistently a segment tree built over the event timeline; have the offline algorithm ready.

Two algorithmic families converge here: (1) offline — sort all events by time, run union-find, snapshot connectivity at each query T; (2) online with unfriend support — vanilla union-find doesn't handle deletion, so either rebuild per query, use link-cut trees, or apply offline reverse-time processing (treat unfriends as adds in reverse). Clarify with the interviewer which they want.

In an L4 R1 variant, the interviewer requested a graph-and-DFS implementation for the unfriend extension, then continued asking for an optimization that reused the earlier union-find work. Completing the rebuild approach did not end that follow-up.

## PREPARATION

Have weighted/path-compressed union-find ready and warm.

Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.

Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

**STUDY HERE: topic 3, Graphs and Search**

Also fits: Strings, Parsing and Recursion (listed there as SKIP, no pages). Site tags: string, parsing, graph, topological-sort, dfs, cycle-detection, medium

**MEDIUM | 1Point3Acres | onsite-coding | 2 of 11**

# Template Variable Expansion with Follow-Ups

**ASKED 2026-10-17 | 3 DAYS AGO | LAST 30 DAYS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 214/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-17 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

## QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

**Input:** a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string.
Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
**Follow-up:** detect cyclic dependencies among map values.
Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

**EXAMPLES**

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

## APPROACH

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

## HOW TO THINK ABOUT IT

Agree the grammar first: %NAME% is a placeholder, %% is a literal percent sign, an unmatched % is an error and an unknown name is an error. Values may contain placeholders themselves, so expand a value by expanding its own placeholders first. A cycle (A uses B, B uses A) would recurse forever, so detect it with a visiting set or, equivalently, with indegrees and a queue.

**Basis:** Written for this question; resolving values that contain other placeholders is a dependency graph problem (as in LeetCode 210, Course Schedule II).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def expand(text, depth=0):
    if depth > LIMIT:
        raise Error("too deep: cycle?")
    out = ""
    for token in tokens(text):
        if token == "%%":
            out += "%"
        elif is_key(token):
            value = values[name_of(token)]
            out += expand(value, depth + 1)
        else:
            out += token
    return out
```

| TIME | SPACE |
|---|---|
| **exponential in the worst ...** exponential in the worst case | **O(depth)** |

**WHY IT IS SLOW**

Re-expands shared values every time and finds cycles only by running out of depth. Memoising each value and keeping a visiting set expands each key once and reports a cycle at once.

## OPTIMAL SOLUTION · Depth-first expansion with memo and a visiting set

| TIME | SPACE |
|---|---|
| **O(total length of the expanded text + keys)** | **O(keys)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Detect cycles before expanding.

Model each key as a vertex with an edge to every key its value mentions, count indegrees and run Kahn's algorithm: if fewer vertices than keys come out, some keys sit on a cycle and cannot be resolved. This reports the problem up front instead of failing halfway through a long document.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict, deque


# HELPERS for the base question
def tokens(s):
    # split text into ("text", char) and ("var", name)
    i = 0
    while i < len(s):
        if s[i] != "%":
            yield "text", s[i]
            i += 1
        elif s[i:i + 2] == "%%":
            # %% is a literal percent sign
            yield "text", "%"
            i += 2
        else:
            j = s.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            yield "var", s[i + 1:j]
            i = j + 1


# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
def expand_template(text, mapping):
    done, visiting = {}, set()

    def resolve(name):
        # values may hold placeholders: expand them first
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        return "".join(resolve(x) if kind == "var" else x
                       for kind, x in tokens(s))
```

```
    return scan(text)


# FOLLOW-UP 1: Detect cycles before expanding.
def has_cycle(mapping):
    # Kahn's algorithm over key -> keys it mentions
    deps = {}
    for k, v in mapping.items():
        deps[k] = {x for kind, x in tokens(v)
                   if kind == "var"}
    users, indeg = defaultdict(list), {}
    for k, d in deps.items():
        known = d & mapping.keys()
        indeg[k] = len(known)
        for x in known:
            users[x].append(k)
    queue = deque(k for k, n in indeg.items() if n == 0)
    resolved = 0
    while queue:
        k = queue.popleft()
        resolved += 1
        for u in users[k]:
            indeg[u] -= 1
            if indeg[u] == 0:
                queue.append(u)
    # keys left over sit on a cycle
    return resolved != len(mapping)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    assert expand_template(text, m) == (
        "I am admin My home is /admin/home")
    sure = expand_template("100%% sure", {})
    assert sure == "100% sure"
    assert not has_cycle(m)
    loop = {"A": "%B%", "B": "%A%"}
    assert has_cycle(loop)
    for bad, error in (("a % b", ValueError),
                       ("%NOPE%", KeyError)):
        try:
            expand_template(bad, {})
            raise AssertionError("should have failed")
        except error:
            pass
    try:
        expand_template("%A%", loop)
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
```

### NOTES FROM CANDIDATES

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

### PREPARATION

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

**STUDY HERE: topic 3, Graphs and Search**

Site tags: graph, bfs, shortest-path, medium

**MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 3 of 11**

# Train / Flight Schedule Reachability

**ASKED 2026-10-08 | 12 DAYS AGO | LAST 30 DAYS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 182/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 2092 Find All People With Secret (related)

Frequency: Low | Last asked: 2026-10-08 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/train-schedule-reachability

## QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

**Input:** a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
**Output:** whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

**EXAMPLES**

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late
    for the Y->B connection, but the second one makes
    it.
```

## APPROACH

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask whether a connection with departure equal to arrival is allowed, whether a leg can arrive at the same time it leaves, whether times are integers, and whether the answer is yes or no or the route.

## INTERVIEW-DAY PLAN

**BRUTE FORCE TO SAY OUT LOUD**

Search over (station, arrival time) states following any leg that departs later. O(legs^2).

**OPTIMAL IDEA**

Sort the legs by departure time and keep earliest[station]. The source starts at minus infinity. For each leg, if earliest[origin] is before (or equal to, if allowed) the departure, set earliest[destination] = min(earliest[destination], arrival). The destination is reachable when it has an entry. O(L log L); the table also gives the earliest arrival time, and a parent leg per station gives the route.

**DRY-RUN THESE CASES**

The example with the late first A to Y leg, source equals destination, no legs, a connection with arrival exactly equal to departure, parallel legs, a cycle in the stations.

**LIKELY FOLLOW-UPS**

Return the route or the earliest arrival, many queries on one schedule (group by origin and use a time-ordered heap), or minimum number of legs.

**PITFALLS**

Ignoring time and treating it as station reachability (the stated trap). Mixing up strict and non-strict connection times. Processing legs in input order instead of departure order.

## HOW TO THINK ABOUT IT

Plain station-to-station reachability is wrong because a leg is only usable if you are already at its departure station before it leaves. Time is part of the state, so ask what you need to know about each station: the earliest time you can be there. Process the legs in order of departure time; a leg is usable if the earliest arrival at its origin is not later than its departure, and then it can improve the earliest arrival at its destination. One sort and one pass answer reachability, and the same table gives the earliest arrival time.

**Basis:** LeetCode 2092, Find All People With Secret, and LeetCode 787, Cheapest Flights Within K Stops: reachability where time is part of the state.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def reachable(legs, src, dst):
    stack, seen = [(src, -INF)], set()
    while stack:
        station, now = stack.pop()
        if station == dst:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for a, b, dep, arr in legs:
            if a == station and dep > now:
                stack.append((b, arr))
    return False
```

## BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(legs^2) | O(legs) |

**WHY IT IS SLOW**

Every state rescans all legs and the same station is reached at many times, O(legs^2). Sorting the legs by departure and keeping only the earliest arrival per station answers in O(L log L).

## OPTIMAL SOLUTION · Sort by departure and keep earliest arrivals

| TIME | SPACE |
|---|---|
| O(L log L) | O(S) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return the earliest arrival time at the destination.

The table already holds it: earliest_arrivals(legs, source)[dest], or infinity when the station is unreachable. That is the same one pass, so it costs nothing extra.

## FOLLOW-UP 2 Print the route, not just yes or no.

Remember the leg that last improved each station's earliest arrival, then walk back from the destination to the source and reverse. Because only a strictly better arrival replaces the stored leg, the stored legs form a valid chain in time.

## FOLLOW-UP 3 Many queries on the same schedule: what changes in the graph representation?

Group the legs by origin station and sort each list by departure time. For a query the earliest arrival is relaxed with a heap ordered by time (Dijkstra on time), and the first leg you may board at a station is found with bisect on its departure list, so each query costs O(L log S) instead of re-sorting every time. If the source is fixed and the queries only differ by destination, keep the earliest-arrival table and answer each in O(1).

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
def earliest_arrivals(legs, source, strict=True):
    """Time is part of the state: keep the earliest time
    we can be at each station. Legs go in departure
    order, so every leg that could feed this one is
    already done."""
    earliest = {source: -INF}
    by_departure = sorted(legs, key=lambda x: x[2])
    for frm, to, dep, arr in by_departure:
        have = earliest.get(frm, INF)
        # we must already be there before it leaves
        if have < dep or (not strict and have == dep):
            if arr < earliest.get(to, INF):
                earliest[to] = arr
    return earliest


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)


# FOLLOW-UP 2: Print the route, not just yes or no.
def itinerary(legs, source, dest, strict=True):
    """Follow-up: the route. Remember the leg that last
    improved each station, then walk back from the end."""
    earliest, came = {source: -INF}, {}
    for leg in sorted(legs, key=lambda x: x[2]):
        frm, to, dep, arr = leg
        have = earliest.get(frm, INF)
        usable = have < dep or (not strict
                                and have == dep)
        if usable and arr < earliest.get(to, INF):
            earliest[to], came[to] = arr, leg
    if dest not in earliest:
        return None
    route = []
    while dest != source:
        leg = came[dest]
        route.append(leg)
        dest = leg[0]
    return route[::-1]


# CHECKER (brute force)
def reachable_brute(legs, source, dest, strict=True):
    # the oracle: search over (station, arrival time)
    # states
    seen, stack = set(), [(source, -INF)]
    while stack:
        station, now = stack.pop()
        if station == dest:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for frm, to, dep, arr in legs:
            ok = dep > now if strict else dep >= now
            if frm == station and ok:
                stack.append((to, arr))
    return False


if __name__ == "__main__":
    legs = [("A", "Y", 1, 20), ("A", "Y", 2, 5),
            ("Y", "B", 6, 9)]
    assert reachable(legs, "A", "B")
    # without the early A to Y train the connection is
    # lost
    assert not reachable(legs[:1] + legs[2:], "A", "B")
    assert earliest_arrivals(legs, "A")["B"] == 9
    assert itinerary(legs, "A", "B") == [legs[1], legs[2]]
    assert itinerary(legs, "B", "A") is None
```

```
assert reachable(legs, "A", "A")
touch = [("A", "B", 1, 5), ("B", "C", 5, 8)]
assert not reachable(touch, "A", "C")
assert reachable(touch, "A", "C", strict=False)
import random
for _ in range(300):
    many = []
    for _ in range(random.randint(0, 8)):
        dep = random.randint(0, 9)
        many.append((random.choice("ABCD"),
                     random.choice("ABCD"), dep,
```

```
                     dep + random.randint(1, 4)))
    src = random.choice("ABCD")
    dst = random.choice("ABCD")
    for strict in (True, False):
        want = reachable_brute(many, src, dst, strict)
        got = reachable(many, src, dst, strict)
        assert got == want
```

**NOTES FROM CANDIDATES**

Pitfalls: Plain station-level reachability is wrong: in the example, A->Y->B exists as a path, but only one of the two A->Y legs makes the connection.
Clarify whether a connection with departure exactly equal to arrival is allowed.

Follow-ups: Follow-ups push on graph representation

**PREPARATION**

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

**STUDY HERE: topic 3, Graphs and Search**

Site tags: graph, topological-sort, cycle-detection, medium

**MEDIUM | IN APP LC 210 | 1Point3Acres | onsite-coding | 4 of 11**

# Course Scheduling and Dependency Variants

**ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 179/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** your app's interview approach (LeetCode #210)

**LeetCode match:** LC 210 Course Schedule II

Frequency: Medium | Last asked: 2026-10-04

**Original:** https://www.1point3acres.com/interview/problems/company/google/course-schedule-dependency-variants

## QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

**Course-ordering variant**

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

**Damaged-node and path variant**

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

**Merge-two-ordered-sequences variant**

**Input:** two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

**EXAMPLES**

**Merge-two-ordered-sequences**

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1:
    contradiction)
```

## APPROACH

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

## HOW TO THINK ABOUT IT

Every 'A must come before B' rule is an edge. A valid ordering exists exactly when the graph has no cycle, and Kahn's algorithm finds one: repeatedly take a vertex with no unmet prerequisites. The two-array merge is the same problem: each adjacent pair inside an array is an edge and equal values are the same vertex; the merge is possible only if the combined graph has no cycle.

**Basis:** LeetCode 210, Course Schedule II, plus a merge-two-ordered-sequences variant (the same graph in disguise).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(n, rules):
    for order in permutations(range(n)):
        pos = {v: i for i, v in enumerate(order)}
        ok = all(pos[a] < pos[b]
                 for a, b in rules)
        if ok:
            return True
    return False
```

| TIME | SPACE |
|---|---|
| O(V! * E) | O(V) |

**WHY IT IS SLOW**

V! orderings. Kahn's algorithm repeatedly takes a vertex with no unmet prerequisites and finishes in O(V + E).

## OPTIMAL SOLUTION · Kahn's algorithm (course order, or can two sequences merge)

| TIME | SPACE |
|---|---|
| O(V + E) | O(V + E) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

### FOLLOW-UP 1 How do you check that an order is valid?

Record each vertex's position in the order, check that every vertex appears exactly once, then check that for every edge (u, v) the position of u is smaller than the position of v. Accept any valid order, not one fixed answer, because independent vertices can swap.

### FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict, deque


# HELPERS for the base question
def edges_from_sequences(*sequences):
    # each neighbouring pair in a sequence is a rule
    return [(s[i], s[i + 1]) for s in sequences
            for i in range(len(s) - 1)]


def merge_two_orders(a, b):
    # shared values are the same vertex
    edges = edges_from_sequences(a, b)
    # a cycle (conflict) gives None
    return topological_order(set(a) | set(b), edges)


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
def topological_order(vertices, edges):
    # edge (u, v) means u must come before v
    adj = defaultdict(list)
    indeg = {v: 0 for v in vertices}
    for u, v in edges:
        adj[u].append(v)
        # count unmet prerequisites of v
        indeg[v] = indeg.get(v, 0) + 1
        indeg.setdefault(u, 0)
    # start with everything that has no prerequisite
    queue = deque(v for v, d in indeg.items() if d == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        # finishing u frees its dependents
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    # fewer than all vertices means a cycle: no order
    return order if len(order) == len(indeg) else None


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
```

```
def can_merge(a, b):
    # True when some order keeps both sequences in order
    return merge_two_orders(a, b) is not None


# FOLLOW-UP 1: How do you check that an order is valid?
def is_valid_order(order, vertices, edges):
    # every vertex must appear exactly once
    if len(order) != len(set(vertices)):
        return False
    if set(order) != set(vertices):
        return False
    pos = {v: i for i, v in enumerate(order)}
    # every prerequisite must come before its dependent
    return all(pos[u] < pos[v] for u, v in edges)


if __name__ == "__main__":
    # course order with an isolated course 3
    courses, rules = range(4), [(0, 1), (1, 2)]
    order = topological_order(courses, rules)
    assert is_valid_order(order, courses, rules)
    # a swapped order breaks a rule
    swapped = [1, 0, 2, 3]
    assert not is_valid_order(swapped, courses, rules)
    # a cycle has no order
    loop = [(0, 1), (1, 0)]
    assert topological_order(range(2), loop) is None
    # merge examples from the question
    assert can_merge([1, 2, 3], [2, 4, 5])
    merged = merge_two_orders([1, 2, 3], [2, 4, 5])
    assert merged is not None
    assert not can_merge([1, 6, 4], [4, 1])
    # a repeated value in one array is a self conflict
    assert not can_merge([1, 2, 1], [])
```

### NOTES FROM CANDIDATES

For standard prerequisite-ordering practice, represent each course as a vertex and each prerequisite as an edge from prerequisite to dependent. Include isolated courses. The result orders every course after its prerequisites; a cycle prevents a complete ordering.

Maintain indegrees and a queue of zero-indegree vertices. Emit a vertex, decrement its outgoing neighbors, and enqueue newly ready vertices. If fewer than V vertices are emitted, report failure under the chosen return convention. This takes O(V + E) time and space; check each edge against positions in the output to validate an ordering.

This ordering algorithm does not determine the nested-list grouping rule or solve the damaged-node path objective. Keep those contracts separate rather than assuming they are ordinary topological sorting.

The merge-two-sequences variant is the same ordering engine in disguise: shared values are the same node, and each adjacent pair inside an array is an edge. A cycle means the merge is impossible. It was asked as the final onsite round of an early-career L3 loop, and the candidate nearly ran out of time — recognizing it as a graph problem early is the main hurdle.

### PREPARATION

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

**STUDY HERE: topic 3, Graphs and Search**

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: grid, dfs, bfs, union-find, medium

**MEDIUM | IN APP LC 200 | 1Point3Acres | onsite-coding, phone-screen | 5 of 11**

**★ TOP 75: rank 17 in the Medium and Hard priority list**

# ★ Number of Islands

**ASKED 2026-09-30 | 20 DAYS AGO | LAST 30 DAYS**

**STUDY: TYPE** **ALL TIME 64.8** **30 DAYS 33** **PRIORITY 19/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** your app's interview approach (LeetCode #200)

**LeetCode match:** LC 200 Number of Islands

Frequency: Medium | Last asked: 2026-09-30 | Roles: SWE, MLE

**Original:** https://www.1point3acres.com/interview/problems/company/google/number-of-islands

## QUESTION

**EXAMPLES**

```
[[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2.
Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1,
    1] (last addition merges the two land cells).
```

## APPROACH

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

## HOW TO THINK ABOUT IT

Each island is a connected group of land cells, so count how many times you start a fresh search at an unvisited land cell. The search itself (DFS or BFS) just marks everything reachable. If land is added over time, do not rerun the search: Union-Find keeps the count up to date.

**Basis:** LeetCode 200, Number of Islands, and LeetCode 305, Number of Islands II (streaming additions with Union-Find).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

## BRUTE FORCE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == 0 or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
def dfs(r, c):
    if out_of_grid(r, c) or grid[r][c] == 0:
        return
    if (r, c) in seen:
        return
    seen.add((r, c))
    for dr, dc in DIRECTIONS:
        dfs(r + dr, c + dc)
```

| TIME | SPACE |
|---|---|
| O(m * n) | O(m * n) recursion |

**WHY IT IS SLOW**

Time is already O(m * n); the weakness is the recursion depth, which can reach m * n on a snake-shaped island and overflow the stack. An explicit queue or stack avoids that.

## OPTIMAL SOLUTION · Iterative DFS flood fill

| TIME | SPACE |
|---|---|
| O(mn) | O(mn) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 What if land can also be removed?

Union-Find cannot split sets. Either rebuild the affected component with a local BFS from each former neighbour, or process the whole timeline offline with a segment tree over time and union-find with rollback (the offline dynamic connectivity technique).

## FOLLOW-UP 2 How do you handle a grid too large for memory?

Process it in horizontal strips: label islands within the strip, keep only the boundary row's labels, and merge labels across strips with a union-find over label ids. Memory is O(width + number of labels on the boundary).

## FOLLOW-UP 3 Find the area of the largest island.

Count the cells during each search and keep the maximum. Same traversal, one counter.

### FOLLOW-UP 4 The grid is huge and does not fit in memory.

Process it in strips of rows, label islands in each strip, and merge labels across strip boundaries with a Union-Find over the labels on the boundary row. **Cost:** O(m * n) time overall and O(width + labels on the boundary row) memory per strip.

### FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# HELPERS for the base question
DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def neighbors(r, c, rows, cols):
    # the cells beside (r, c) that are inside the grid
    for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            yield nr, nc


def flood(grid, start, seen):
    # visit all land joined to start; return the count
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack, size = [start], 0
    # a stack instead of recursion: no recursion limit
    while stack:
        r, c = stack.pop()
        size += 1
        for nb in neighbors(r, c, rows, cols):
            if grid[nb[0]][nb[1]] == 1 and nb not in seen:
                seen.add(nb)
                stack.append(nb)
    return size


def island_sizes(grid):
    if not grid:
        return []
    seen, sizes = set(), []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            # an island starts at land not visited yet
            if grid[r][c] == 1 and (r, c) not in seen:
                sizes.append(flood(grid, (r, c), seen))
    return sizes


# BASE QUESTION (optimal): Iterative DFS flood fill
def num_islands(grid):
    # one entry per island
    return len(island_sizes(grid))
```

```
# VARIANT: land cells arrive one at a time
class IslandCounter:
    """Follow-up: land cells arrive one at a time."""

    def __init__(self, rows, cols):
        self.rows, self.cols = rows, cols
        # land cell -> parent cell (union-find)
        self.parent = {}
        self.count = 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_land(self, r, c):
        # adding the same cell twice changes nothing
        if (r, c) in self.parent:
            return self.count
        self.parent[(r, c)] = (r, c)
        self.count += 1
        # join with every land neighbour
        for nb in neighbors(r, c, self.rows, self.cols):
            if nb in self.parent:
                a, b = self.find((r, c)), self.find(nb)
                if a != b:
                    self.parent[a] = b
                    # two islands merged into one
                    self.count -= 1
        return self.count


# FOLLOW-UP 3: Find the area of the largest island.
def largest_island(grid):
    # follow-up: the area of the biggest island
    return max(island_sizes(grid), default=0)


if __name__ == "__main__":
    g = [[1, 1, 0, 0], [1, 1, 0, 0],
         [0, 0, 1, 1], [0, 0, 1, 1]]
    assert num_islands(g) == 2
```

```
    assert largest_island(g) == 4
    assert num_islands([]) == 0
    # streaming adds: counts stay 1, 1, 1
    ic = IslandCounter(2, 2)
    adds = [(0, 0), (0, 1), (1, 1)]
    counts = [ic.add_land(r, c) for r, c in adds]
    assert counts == [1, 1, 1]
    # two separate cells make two islands
    ic = IslandCounter(3, 3)
    assert ic.add_land(0, 0) == 1
    assert ic.add_land(2, 2) == 2
    assert ic.add_land(1, 1) == 3
```

### NOTES FROM CANDIDATES

For streaming, Union-Find with path compression is the natural choice — each addLand is amortized alpha(n).
In one onsite, the follow-up arrived after a working BFS solution and explicitly required a Union-Find rewrite; retrofitting the original traversal consumed the remaining time.
Common edge cases: diagonal connectivity (some prompts use 8-connectivity), grid with no land.

### PREPARATION

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

**STUDY HERE: topic 3, Graphs and Search**

Also fits: Heaps, Stacks and Queues, Greedy and Math (listed there as SKIP, no pages). Site tags: graph, heap, algorithm, medium

**MEDIUM | IN APP LC 778 | 1Point3Acres | onsite-coding | 6 of 11**

# Path Minimizing the Maximum Cell (Swim in Rising Water)

**ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS**

**STUDY: TYPE** **ALL TIME 40** **30 DAYS 33** **PRIORITY 55/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** your app's interview approach (LeetCode #778)

**LeetCode match:** LC 778 Swim in Rising Water

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

## QUESTION

A bottleneck-path family: minimize the maximum value encountered on a grid path, then generalize the same reasoning to a weighted graph whose edges carry safety values and an admissibility threshold.

**Grid variant**

Input is a grid of heights and start/end cells; movement is through four-neighbors.
Cost of a path is the maximum cell value along it.
Return the minimum possible path cost.

**Weighted-graph safety variant**

Input is a graph whose edges carry safety values, a start node, an end node, and a safety threshold.
Find the lowest safety value required by any valid start-to-end path while every traversed edge remains under the supplied threshold.
Return -1 when no admissible path exists.
Clarify whether the threshold comparison is strict or inclusive before coding.

**Mouse-and-cat safest-path variant (L4 phone screen)**

Warm-up: an N x N grid with a start S, a target T, and impassable water cells; with 4-directional moves, decide whether any path from S to T exists.
Main question: the grid also contains a cat at C. The mouse moves only on land, 4-directionally, from S to the cheese at T. Find a path that maximizes the minimum Manhattan distance to the cat over all cells on the path.
This flips the objective (maximize a minimum instead of minimize a maximum), but it is the same bottleneck-path structure once each cell's distance to the cat is known.

**Follow-up:** Network Delay Time variant

In a directed graph with edge weights, return the maximum shortest-path distance from a source to all nodes, or -1 if any node is unreachable.

## QUESTION

**EXAMPLES**
**Grid**

```
[ 0  1  2  3  4 ]
[24 23 22 21  5 ]
[12 13 14 15 16 ]
[11 17 18 19 20 ]
[10  9  8  7  6 ]
```

Start=(0,0), End=(4,0) in (row, col) — the bottom-left cell with value 10. Answer: 16 (the path's max cell is 16).

## APPROACH

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

## HOW TO THINK ABOUT IT

The cost of a path is its largest cell, not the sum. That cost can only grow as the path grows, so Dijkstra still works if you replace 'distance + weight' by max(distance, weight): the first time a cell leaves the heap its cost is final. The cat version flips the objective (maximise a minimum), which is the same shape with a max-heap and each cell scored by its distance to the cat.

**Basis:** LeetCode 778, Swim in Rising Water (bottleneck path), plus the 'safest path from a cat' flip: maximise the minimum Manhattan distance.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(grid, start, end):
    for t in sorted(set(all_values(grid))):
        if bfs_reaches(grid, start, end, t):
            return t  # smallest limit
```

| TIME | SPACE |
|---|---|
| O(V^2) | O(V) |

**WHY IT IS SLOW**

One full BFS per distinct value, O(V^2). Dijkstra on the running maximum finds the answer in one pass, O(V log V).

## OPTIMAL SOLUTION · Dijkstra on the running maximum

| TIME | SPACE |
|---|---|
| O(V log V) | O(V) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 Maximise the minimum distance to a cat instead.**

Same bottleneck shape with the objective flipped: score each cell by its Manhattan distance to the cat, run Dijkstra with a max-heap on the running minimum, and skip water. The same monotonicity argument holds (extending a path can only lower its minimum).

**FOLLOW-UP 2 Why is Dijkstra correct for a max-along-path cost?**

The cost max(path) never decreases when a path is extended, so a vertex's cheapest cost cannot be improved by going through a vertex that is popped later. That is the only property Dijkstra needs from its cost function (monotone, non-negative extension).

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq


# HELPERS for the base question
INF = float("inf")


def around(r, c, n, m):
    # four-neighbour cells inside an n x m grid
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < n and 0 <= b < m:
            yield a, b


# BASE QUESTION (optimal): Dijkstra on the running maximum
def min_max_path(g, start, end):
    n, m = len(g), len(g[0])
    # best[cell] = smallest possible "highest cell so far"
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, (r, c) = heapq.heappop(heap)
        # skip outdated heap entries
        if cost > best[(r, c)]:
            continue
        if (r, c) == end:
            return cost
        for a, b in around(r, c, n, m):
            # path cost = highest cell so far or this cell
            nxt = max(cost, g[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, (a, b)))
    return None


# VARIANT: weighted graph with a safety threshold
def min_bottleneck_edge(graph, s, t, limit):
    # graph[u] = [(v, safety)]; every edge must be < limit
    best = {s: 0}
    heap = [(0, s)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:
            continue
        if u == t:
            return cost
        for v, w in graph.get(u, []):
            # edges at or above the limit are not allowed
            if w >= limit:
                continue
            nxt = max(cost, w)
            if nxt < best.get(v, INF):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    # no admissible path
    return -1


# RELATED: network delay (plain Dijkstra)
def network_delay(n, edges, src):
    # plain Dijkstra: largest shortest distance, or -1
    graph = {}
    for u, v, w in edges:
        graph.setdefault(u, []).append((v, w))
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    # unreachable nodes make the answer -1
    return max(dist.values()) if len(dist) == n else -1


# FOLLOW-UP 1: Maximise the minimum distance to a cat
# instead.
def safest_path_score(grid, start, target, cat):
    # maximise the smallest cat distance ('#' is water)
    n, m = len(grid), len(grid[0])

    def dist(r, c):
        return abs(r - cat[0]) + abs(c - cat[1])

    best = {start: dist(*start)}
    heap = [(-best[start], start)]
    while heap:
        neg, (r, c) = heapq.heappop(heap)
        if -neg < best[(r, c)]:
            continue
        if (r, c) == target:
            return -neg
        for a, b in around(r, c, n, m):
            if grid[a][b] == "#":
                continue
            # the path is only as safe as its closest cell
            score = min(-neg, dist(a, b))
            if score > best.get((a, b), -1):
                best[(a, b)] = score
                heapq.heappush(heap, (-score, (a, b)))
    return None


if __name__ == "__main__":
    g = [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5],
         [12, 13, 14, 15, 16], [11, 17, 18, 19, 20],
         [10, 9, 8, 7, 6]]
    assert min_max_path(g, (0, 0), (4, 0)) == 16
    # start equals end: just that cell
```

```
assert min_max_path(g, (2, 2), (2, 2)) == 14
gr = {0: [(1, 5), (2, 9)], 1: [(2, 3)]}
assert min_bottleneck_edge(gr, 0, 2, 10) == 5
assert min_bottleneck_edge(gr, 0, 2, 5) == -1
cat_grid = ["...", ".#.", "..."]
cat, a, b = (0, 2), (0, 0), (2, 2)
score = safest_path_score(cat_grid, a, b, cat)
assert score == 2
chain = [(0, 1, 2), (1, 2, 3)]
assert network_delay(3, chain, 0) == 5
assert network_delay(3, [(0, 1, 2)], 0) == -1
```

## NOTES FROM CANDIDATES

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter.

Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

In the mouse-and-cat variant, each cell's score is a direct Manhattan distance to a single cat, so it can be computed per cell; clarify whether water cells block only the mouse or also affect distance, and whether S or T themselves count toward the minimum.

## PREPARATION

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

**STUDY HERE: topic 3, Graphs and Search**

Site tags: graph, bfs, shortest-path, medium

**MEDIUM | 1Point3Acres | onsite-coding | 7 of 11**

# Shortest Path Nodes and Escaping a Pursuer

**ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS**

**STUDY: READ** **NO LEETCODE FREQUENCY** **PRIORITY 212/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/shortest-path-nodes-and-pursuer-escape

## QUESTION

Unweighted undirected graph: shortest path from Alice's node to a target (BFS). Follow-up 1: all nodes that lie on at least one shortest path. Follow-up 2: Bob starts at another node at the same time; decide whether Alice can follow some shortest path to the target without being caught.

**Input:** an undirected, unweighted graph; Alice starts at one node and must reach a target node.
Return the shortest path (length) from Alice to the target.
**Follow-up 1:** find all nodes that can appear on some shortest path from Alice to the target.
**Follow-up 2:** Bob is at another node. Alice picks one shortest path and starts moving; Bob starts moving at the same time. Decide whether Alice can reach the target safely without being caught by Bob.

## APPROACH

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target]. For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

## HOW TO THINK ABOUT IT

BFS from Alice gives dA and BFS from the target gives dT. A node is on some shortest path exactly when dA[v] + dT[v] equals the shortest length. For the pursuer, BFS from Bob gives dB. State the rule you assume: Alice is caught at a node v if Bob can be there no later than she is (dB[v] <= dA[v]). Alice is safe when a shortest path exists whose every node has dB[v] > dA[v].

## HOW TO THINK ABOUT IT

**Basis:** Written for this question (BFS layers, as in LeetCode 1976, Number of Ways to Arrive at Destination).

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(graph, alice, bob, target):
    dA = bfs_dist(graph, alice)
    dB = bfs_dist(graph, bob)
    paths = all_paths(graph, alice, target)
    for path in paths:
        if len(path) - 1 != dA[target]:
            continue  # not a shortest path
        if all(dB[v] > dA[v] for v in path):
            return True  # Bob is always late
    return False
```

| TIME | SPACE |
|---|---|
| exponential in the worst ... exponential in the worst case | O(paths) |

**WHY IT IS SLOW**

The number of shortest paths can be exponential. Three BFS runs (from Alice, the target and Bob) decide every node in O(V + E).

## OPTIMAL SOLUTION · Three BFS runs and a safe shortest-path DAG

| TIME | SPACE |
|---|---|
| O(V + E) | O(V) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 What if Bob can also meet Alice in the middle of an edge?

Then arriving at the same node at the same time is not the only way to be caught: Alice moving u -> v while Bob moves v -> u swaps places. Add that case by also rejecting an edge (u, v) when dB[v] == dA[u] and dB[u] == dA[v]. Always clarify what 'caught' means first.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import deque


# HELPERS for the base question
FAR = 10 ** 9
```

```
def bfs(adj, s):
    # distance from s to every reachable node
    d = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in d:
                d[v] = d[u] + 1
                q.append(v)
    return d


# BASE QUESTION (optimal): Three BFS runs and a safe
# shortest-path DAG
def shortest_length(adj, a, t):
    # the base question: edges on the shortest path
    return bfs(adj, a).get(t)


# QUESTION FOLLOW-UP 1: all nodes on a shortest path
def nodes_on_shortest_paths(adj, a, t):
    da, dt = bfs(adj, a), bfs(adj, t)
    if t not in da:
        return set()
    # a node is on some shortest path if the two
    # distances add up to the full length
    return {v for v in da if v in dt
            and da[v] + dt[v] == da[t]}


# QUESTION FOLLOW-UP 2: can Alice escape Bob?
def can_escape(adj, a, t, b, meet_on_edge=False):
    da, db = bfs(adj, a), bfs(adj, b)
    if t not in da:
        return False
    on_path = nodes_on_shortest_paths(adj, a, t)
```

```
    def safe(u, v):
        # Bob must reach v strictly after Alice does
        if db.get(v, FAR) <= da[v]:
            return False
        # FOLLOW-UP 1: meet_on_edge=True: they meet inside
        # an edge
        # optional: they swap places on the edge u-v
        swap = db.get(v, FAR) == da[u]
        swap = swap and db.get(u, FAR) == da[v]
        return not (meet_on_edge and swap)

    # Bob standing on Alice's start catches her at once
    if db.get(a, FAR) <= 0:
        return False
    # walk the shortest-path layers using safe nodes only
    frontier, seen = [a], {a}
    while frontier:
        nxt = []
        for u in frontier:
            for v in adj[u]:
                if v in on_path and da[v] == da[u] + 1 \
                        and v not in seen and safe(u, v):
                    seen.add(v)
                    nxt.append(v)
        frontier = nxt
    return t in seen


if __name__ == "__main__":
    # a square 0-1-3 and 0-2-3 plus a tail 3-4
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3],
           3: [1, 2, 4], 4: [3]}
    assert shortest_length(adj, 0, 4) == 3
    on_path = nodes_on_shortest_paths(adj, 0, 4)
    assert on_path == {0, 1, 2, 3, 4}
```

```
    # Bob waiting at the target catches Alice there
    assert not can_escape(adj, 0, 4, 4)
    chain = {0: [1], 1: [0, 2], 2: [1, 3], 3: [2]}
    # Bob starts at the target and reaches it first
    assert not can_escape(chain, 0, 3, 3)
    # Bob starts behind Alice: she stays ahead
    assert can_escape(chain, 1, 3, 0)
```

## NOTES FROM CANDIDATES

Asked as an L4 onsite coding round.
Follow-up 2 has an open definition of "caught": clarify whether it means arriving at the same node at the same time, Bob arriving first, or also meeting on an edge; also clarify whether Alice may choose her shortest path knowing Bob's position.

## PREPARATION

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target].
For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

**STUDY HERE: topic 3, Graphs and Search**

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: grid, bfs, medium

**MEDIUM | IN APP LC 994 | 1Point3Acres | onsite-coding, phone-screen | 8 of 11**

# Rotten Oranges / Multi-Source BFS (taxis)

**ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS**

**STUDY: TYPE** **ALL TIME 51.2** **30 DAYS 0** **PRIORITY 67/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** written and tested here for this exact question

This question has several parts, so each part has its own answer below.

**Also in your app:** LeetCode #994 Rotting Oranges

**LeetCode match:** LC 994 Rotting Oranges

Frequency: Low | Last asked: 2026-09-08 | Roles: SWE, MLE

**Original:** https://www.1point3acres.com/interview/problems/company/google/rotten-oranges-multi-source-bfs-taxis

## QUESTION

**EXAMPLES**

```
Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges
    variant): return number of minutes until all
    reachable 1s become 2 (4).
```

Taxi variant: for each empty cell, distance to nearest taxi.

## APPROACH

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

## HOW TO THINK ABOUT IT

'Spread outward from several starting points at the same time and count the rounds' is multi-source BFS: put every source in the queue before you start, and process the queue one round at a time. Rounds are minutes. Always count the fresh ones first so that unreachable oranges are detected at the end.

**Basis:** LeetCode 994, Rotting Oranges, and the same multi-source BFS skeleton as LeetCode 542 and 286.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

## BRUTE FORCE

```
def brute(grid):
    minutes = 0
    while True:
        fresh = fresh_next_to_rotten(grid)
        if not fresh:
            break
        for r, c in fresh:
            grid[r][c] = 2
        minutes += 1
    return -1 if any_fresh(grid) else minutes
```

| TIME | SPACE |
|---|---|
| O(minutes * m * n) | O(m * n) |

**WHY IT IS SLOW**

Each minute scans the entire grid, O(minutes * m * n). A queue of the newly rotten oranges touches each cell only once, O(m * n).

## OPTIMAL SOLUTION · Multi-source BFS by minute

| TIME | SPACE |
|---|---|
| O(mn) | O(mn) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## OPTIMAL SOLUTION · Taxi variant: distance from every empty cell to the nearest taxi

| TIME | SPACE |
|---|---|
| O(mn) | O(mn) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

## FOLLOW-UP 2 Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

### FOLLOW-UP 3 Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

### FOLLOW-UP 4 A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

### FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import deque


# HELPERS for the base question
def multi_source_bfs(rows, cols, sources, open_cell):
    # distance from the nearest source to every cell
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c),
                       (r, c + 1), (r, c - 1)):
            inside = 0 <= nr < rows and 0 <= nc < cols
            if inside and (nr, nc) not in dist:
                if open_cell(nr, nc):
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
    return dist


# BASE QUESTION (optimal): Multi-source BFS by minute
def rotting_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    cells = [(r, c) for r in range(rows)
             for c in range(cols)]
    rotten = [x for x in cells if grid[x[0]][x[1]] == 2]
    fresh = [x for x in cells if grid[x[0]][x[1]] == 1]
    # rot spreads through any non-empty cell
    dist = multi_source_bfs(
        rows, cols, rotten, lambda r, c: grid[r][c] != 0)
    # a fresh orange that is never reached means -1
    if any(x not in dist for x in fresh):
        return -1
    return max((dist[x] for x in fresh), default=0)


# TAXI VARIANT: distance to the nearest taxi
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked
    rows, cols = len(grid), len(grid[0])
    taxis = [(r, c) for r in range(rows)
             for c in range(cols) if grid[r][c] == "T"]
    dist = multi_source_bfs(
        rows, cols, taxis, lambda r, c: grid[r][c] != "#")
    # None where no taxi can reach the cell
    return [[dist.get((r, c)) for c in range(cols)]
            for r in range(rows)]


# FOLLOW-UP 4: A signal spreads from sources to targets:
# how long until every target ...
def signal_time(rows, cols, sources, targets, blocked=()):
    # how long until every target hears the signal?
    dist = multi_source_bfs(
        rows, cols, list(sources),
        lambda r, c: (r, c) not in blocked)
    if any(t not in dist for t in targets):
        return -1
    # no targets means no waiting
    return max((dist[t] for t in targets), default=0)


if __name__ == "__main__":
    assert rotting_minutes([[2, 1, 1], [1, 1, 0],
                            [0, 1, 1]]) == 4
    # a fresh orange walled off by an empty cell
    assert rotting_minutes([[2, 0, 1]]) == -1
    assert rotting_minutes([[0, 2]]) == 0
    assert nearest_taxi(["T.#", "..."]) == [
        [0, 1, None], [1, 2, 3]]
    # signal from two sources to two targets
    assert signal_time(1, 5, [(0, 0), (0, 4)],
                       [(0, 1), (0, 2)]) == 2
    assert signal_time(1, 3, [(0, 0)], [(0, 2)],
                       blocked={(0, 1)}) == -1
    assert signal_time(1, 3, [(0, 0)], []) == 0
```

### NOTES FROM CANDIDATES

BFS from all sources simultaneously is O(mn); per-source BFS is O(mn × #sources).
The moving-source follow-up trades exactness for amortized cost — common answer is periodic rebuild with bounded staleness.
For the signal variant, use the maximum source-distance over targets as the completion time; an unreachable target makes that candidate infeasible. Agree that no targets requires zero time. A correct baseline for destroying at most one blocker is to run multi-source BFS with no deletion, then once with each blocker made traversable, restoring it before the next trial. Minimize the completion time across feasible trials; return -1 if none is feasible. With B blockers on an R by C grid, this takes O((B + 1) * R * C) time and O(R * C) reusable auxiliary space. A BFS state that allows each path to break its own blocker does not enforce one shared destroyed blocker for all targets.

### PREPARATION

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

**STUDY HERE: topic 3, Graphs and Search**

Site tags: graph, dfs, bfs, union-find, clustering, medium

**MEDIUM | 1Point3Acres | phone-screen | 9 of 11**

# 2D Points Connected Components by Radius

**ASKED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS**

**STUDY: TALK** **ALL TIME 40.7** **30 DAYS 0** **PRIORITY 91/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #547 Number of Provinces

**LeetCode match:** LC 547 Number of Provinces (related)

Frequency: Low | Last asked: 2026-08-04 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

## QUESTION

Given 2D points and radius `r`, connect two points when their distance is at most `r`; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

**Input:** a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

## APPROACH

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

## HOW TO THINK ABOUT IT

Two points are neighbours when their squared distance is at most r squared (no square roots needed), and connectivity is transitive, so this is counting connected components: union-find or a DFS over the neighbour relation. Checking every pair is O(n^2); bucketing points into cells of side r means a point only needs to be compared with the 3 x 3 cells around it.

**Basis:** LeetCode 547, Number of Provinces, where two points are linked when their distance is at most r.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(points, r):
    graph = defaultdict(list)
    ids = range(len(points))
    for i, j in combinations(ids, 2):
        if dist2(points[i], points[j]) <= r * r:
            graph[i].append(j)
            graph[j].append(i)
    seen, clusters = set(), 0
    for i in range(len(points)):
        if i not in seen:
            clusters += 1
            dfs(graph, i, seen)
    return clusters
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**WHY IT IS SLOW**

Every pair is compared, O(n^2). Putting points in cells of side r means each point is compared only with the 3 x 3 cells around it.

## OPTIMAL SOLUTION · Union-find with a spatial grid

| TIME | SPACE |
|---|---|
| O(n)<br>about when points are spread out<br>(O(n^2) if they all share a cell) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why compare squared distances?

sqrt is slow and inexact; dist <= r is the same as dist^2 <= r^2 for non-negative numbers, and with integer coordinates the comparison is exact. It also avoids floating-point surprises right at the boundary.

## FULL INTERVIEW CODE · base question + every follow-up

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict


# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
```

```
    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
# FOLLOW-UP 1: Why compare squared distances?
def near(p, q, r):
    # squared distance: no square root, exact for ints
    dx, dy = p[0] - q[0], p[1] - q[1]
    return dx * dx + dy * dy <= r * r


# BASELINE: compare every pair
def count_clusters_pairs(points, r):
    # the accepted baseline: compare every pair, O(n^2)
    uf = UnionFind(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if near(points[i], points[j], r):
                uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


# BASE QUESTION (optimal): Union-find with a spatial grid
def count_clusters(points, r):
    # grid of cells r wide: only 9 cells need a look
    size = r if r > 0 else 1
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
```

```
        cells[(x // size, y // size)].append(i)
    uf = UnionFind(len(points))
    for i, (x, y) in enumerate(points):
        cx, cy = x // size, y // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                cell = cells.get((cx + dx, cy + dy), ())
                for j in cell:
                    q = points[j]
                    if j > i and near((x, y), q, r):
                        uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


if __name__ == "__main__":
    # three points joined only through the middle one
    line = [(0, 0), (3, 0), (6, 0)]
    assert count_clusters(line, 3) == 1
    assert count_clusters_pairs(line, 3) == 1
    # the far point stays alone
    far = [(0, 0), (3, 0), (7, 0)]
    assert count_clusters(far, 3) == 2
    # radius 0 only joins identical points
    same = [(1, 1), (1, 1), (2, 2)]
    assert count_clusters(same, 0) == 2
    assert count_clusters([], 5) == 0
    # both methods agree on a scatter of points
    pts = [(i * 7 % 13, i * 5 % 11) for i in range(30)]
    for r in (0, 1, 2, 3, 5):
        assert count_clusters(pts, r) == (
            count_clusters_pairs(pts, r))
```

## NOTES FROM CANDIDATES

DFS, BFS, or Union-Find all fit the core connected-components model.
For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.
Clarify squared-distance comparison to avoid unnecessary square roots.

## PREPARATION

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

**STUDY HERE: topic 3, Graphs and Search**

Site tags: graph, bfs, dijkstra, medium

**MEDIUM | 1Point3Acres | onsite-coding | 10 of 11**

# Minimum Direction Violations in a Directed Graph

**ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS**

**STUDY: TALK** **ALL TIME 10.6** **30 DAYS 0** **PRIORITY 165/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 1466 Reorder Routes to Make All Paths Lead to the City Zero (related)

Frequency: Single report | Last asked: 2026-07-19 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/minimum-direction-violations-directed-graph

## QUESTION

**EXAMPLES**

```
n = 5
edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
start = 0
end = 4

path: 0 -> 1 -> 2 -> 3 -> 4
cost: 0 + 1 + 0 + 1 = 2
output: 2
```

## APPROACH

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask what a violation is (traversing an edge against its direction), whether edge costs are all 1, what to return when the end cannot be reached, and whether the graph has parallel edges or self loops.

**BRUTE FORCE TO SAY OUT LOUD**

Plain BFS ignoring direction minimizes edges, not violations, so say why it is wrong. Dijkstra with costs 0 and 1 works at O((n + m) log n).

**OPTIMAL IDEA**

Add every edge twice: forward with weight 0, backward with weight 1. Run 0-1 BFS with a deque: relax a weight-0 edge by pushing to the front, a weight-1 edge to the back. The deque always holds nodes whose distances differ by at most 1, so O(n + m).

## INTERVIEW-DAY PLAN

**DRY-RUN THESE CASES**

Start equals end (0). A graph with no path. A path that needs every edge reversed. A shortcut that costs more edges but fewer violations.

**LIKELY FOLLOW-UPS**

General edge costs (Dijkstra), shortest path itself (parent pointers), or negative costs (Bellman-Ford).

**PITFALLS**

Using visited flags instead of a distance check. Pushing weight-0 edges to the back.

## HOW TO THINK ABOUT IT

Going against an edge costs 1 and going with it costs 0, so it is a shortest-path problem with costs of only 0 and 1. Plain BFS counts edges, not violations, so it is the wrong tool. A deque lets 0-cost moves jump the queue (push front) and 1-cost moves wait their turn (push back), which is Dijkstra without the heap.

**Basis:** Related to LeetCode 1466, Reorder Routes: a shortest path where traveling an edge backwards costs 1.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
def brute(n, edges, start, end):
    arcs = []
    for u, v in edges:
        arcs.append((u, v, 0))  # along the edge
        arcs.append((v, u, 1))  # against it
    dist = [INF] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[end]
```

| TIME | SPACE |
|---|---|
| O(V * E) | O(V) |

**WHY IT IS SLOW**

Every round scans all edges, V - 1 rounds: O(V * E). Costs of only 0 and 1 allow a deque search (0-1 BFS) in O(V + E).

## OPTIMAL SOLUTION · 0-1 BFS on the doubled graph

| TIME | SPACE |
|---|---|
| O(n + m) | O(n + m) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 Why is 0-1 BFS correct, and how does it compare with Dijkstra?**

The deque always holds nodes whose distances differ by at most one: front elements are at distance d, back elements at d + 1. Relaxing a zero-weight edge keeps the distance at d (front), a one-weight edge makes d + 1 (back). That is the same order a heap would produce, so it is Dijkstra without the log factor: O(n + m) versus O((n + m) log n).

**FOLLOW-UP 2 What if edge costs can be arbitrary non-negative numbers?**

Use Dijkstra with a heap on the same doubled graph (forward cost = 0 or the edge cost, backward cost = the violation cost). With negative costs use Bellman-Ford and watch for negative cycles.

**FOLLOW-UP 3 Edge costs are arbitrary non-negative numbers (different for each direction).**

Use Dijkstra with a heap on the same doubled graph, giving each arc its own cost. 0-1 BFS only works because the costs are 0 or 1.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import deque


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): 0-1 BFS on the doubled graph
def min_violations(n, edges, start, end):
    # 0-1 BFS: following an edge is free, reversing costs
    # 1
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # cost 0 goes to the front, cost 1 to the
                # back
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return -1 if dist[end] == INF else dist[end]


# FOLLOW-UP 3: Edge costs are arbitrary non-negative
# numbers (different for each ...
def cheapest_route(n, edges, start, end):
    # edges are (u, v, forward_cost, backward_cost)
    graph = [[] for _ in range(n)]
    for u, v, fwd, back in edges:
        graph[u].append((v, fwd))
        graph[v].append((u, back))
    dist = [INF] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
        d, u = heapq.heappop(heap)
        # skip outdated heap entries
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return -1 if dist[end] == INF else dist[end]


if __name__ == "__main__":
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    # path 0 -> 1 -> 2 -> 3 -> 4 reverses two edges
    assert min_violations(5, edges, 0, 4) == 2
    assert min_violations(5, edges, 0, 0) == 0
    assert min_violations(3, [(0, 1)], 0, 2) == -1
    # the same graph with every reversal costing 1
    costed = [(u, v, 0, 1) for u, v in edges]
    assert cheapest_route(5, costed, 0, 4) == 2
    # different cost per direction
    assert cheapest_route(2, [(0, 1, 5, 2)], 0, 1) == 5
    assert cheapest_route(2, [(0, 1, 5, 2)], 1, 0) == 2
```

**NOTES FROM CANDIDATES**

Represent each original edge twice: the forward direction with weight 0 and the reverse direction with weight 1.
A regular BFS minimizes the number of edges, not the number of direction violations.
The linear-time target is O(n + m) time and space.

**PREPARATION**

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

**STUDY HERE: topic 3, Graphs and Search**

Site tags: graph, bfs, medium

**MEDIUM | 1Point3Acres | onsite-coding | 11 of 11**

# Shortest Directed Cycle Through a Node

**STUDY: TALK** **ALL TIME 5** **30 DAYS 0** **PRIORITY 174/228**

## ABOUT THIS QUESTION

**Topic:** Graphs and Search

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 2608 Shortest Cycle in a Graph (related)

Frequency: Single report | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/shortest-cycle-through-node

## QUESTION

**EXAMPLES**

```
Edges [(0,1),(1,2),(2,0)], v = 0 -> 3.
Edges [(0,1),(1,2)], v = 0 -> -1.
```

## APPROACH

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).

## INTERVIEW-DAY PLAN

**CLARIFY FIRST**

Ask whether edges are directed (yes), whether self loops count (cycle of length 1), what to return if there is none (-1), and whether edges have weights.

**BRUTE FORCE TO SAY OUT LOUD**

Enumerate all simple paths from v back to v with DFS. Exponential.

**OPTIMAL IDEA**

BFS from v. The first time you see an edge from a visited node u back to v, the cycle length is dist[u] + 1, and BFS order makes it the shortest. Do not mark v as discovered before checking for re-entry. O(V + E).

**DRY-RUN THESE CASES**

A triangle (3). A path with no return (-1). A self loop (1). Two cycles through v of different lengths.

**LIKELY FOLLOW-UPS**

Shortest cycle in the whole graph (BFS from every node), weighted edges (Dijkstra), or undirected graphs.

**PITFALLS**

Marking v visited at the start, which hides the cycle. Counting nodes instead of edges.

## HOW TO THINK ABOUT IT

A cycle through v is a path from v back to v, so find shortest distances from v and watch for an edge that points back at v. Breadth-first search discovers nodes in order of distance, so the first edge back to v closes the shortest cycle. Do not mark v as 'seen' in a way that hides the edge that returns to it.

**Basis:** Related to LeetCode 2608, Shortest Cycle in a Graph, but directed and anchored at a given node.

## BRUTE FORCE

**BRUTE FORCE · PSEUDO-CODE**

```
best = INF
def dfs(u, length, on_path):
    global best
    for w in graph[u]:
        if w == v:
            best = min(best, length + 1)
        elif w not in on_path:
            dfs(w, length + 1, on_path | {w})
dfs(v, 0, {v})
answer = -1 if best == INF else best
```

| TIME | SPACE |
|---|---|
| exponential | O(V) |

**WHY IT IS SLOW**

The number of simple paths can be exponential in the graph size. A breadth-first search from v reaches the shortest cycle in O(V + E) because it visits nodes in order of distance.

## OPTIMAL SOLUTION · BFS from v, close the cycle on the first edge back to v

| TIME | SPACE |
|---|---|
| O(V + E) | O(V) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 How do you find the shortest cycle in the whole graph?

Run the same BFS from every vertex and take the minimum: O(V * (V + E)). For undirected graphs, BFS from each node and treat any non-tree edge as closing a cycle of length dist[u] + dist[w] + 1 (the LeetCode 2608 version).

## FOLLOW-UP 2 What if edges have positive weights?

Replace BFS with Dijkstra from v and close the cycle at the cheapest dist[u] + weight(u, v). That is O((V + E) log V) with a heap. With negative weights use Bellman-Ford and handle negative cycles separately.

**FOLLOW-UP 3 The shortest cycle anywhere in the graph (the girth).**

Run the same BFS from every node and take the smallest result: O(V * (V + E)). That is the standard way and fine for graphs up to a few thousand nodes.

**FULL INTERVIEW CODE · base question + every follow-up**

**One program for the whole question:** the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import deque


# HELPERS for the base question
def make_graph(n, edges):
    adj = [[] for _ in range(n)]
    for a, b, *w in edges:
        # edges may carry a weight (default 1)
        adj[a].append((b, w[0] if w else 1))
    return adj


# HELPERS for follow-up 2
INF = float("inf")


# BASE QUESTION (optimal): BFS from v, close the cycle on
# the first edge back to v
def shortest_cycle_through(n, edges, v):
    # BFS from v: the first edge that comes back to v
    # closes
    # the shortest cycle through v
    adj = make_graph(n, edges)
    dist, queue = {v: 0}, deque([v])
    while queue:
        u = queue.popleft()
        for w, _ in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1


# FOLLOW-UP 1, 3: How do you find the shortest cycle in
# the whole graph?
def girth(n, edges):
    # the shortest cycle anywhere: BFS from every node
    sizes = [shortest_cycle_through(n, edges, v)
             for v in range(n)]
    found = [s for s in sizes if s > 0]
    return min(found) if found else -1
```

```
# FOLLOW-UP 2: What if edges have positive weights?
def weighted_cycle_through(n, edges, v):
    # positive weights: Dijkstra, then close the cycle at
    # v
    adj = make_graph(n, edges)
    dist, heap, best = {v: 0}, [(0, v)], INF
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, INF):
            continue
        for w, cost in adj[u]:
            if w == v:
                best = min(best, d + cost)
            elif d + cost < dist.get(w, INF):
                dist[w] = d + cost
                heapq.heappush(heap, (d + cost, w))
    return -1 if best == INF else best


if __name__ == "__main__":
    ring = [(0, 1), (1, 2), (2, 0)]
    assert shortest_cycle_through(3, ring, 0) == 3
    path = [(0, 1), (1, 2)]
    assert shortest_cycle_through(3, path, 0) == -1
    # a self loop is a cycle of length 1
    assert shortest_cycle_through(2, [(0, 0)], 0) == 1
    # a shorter cycle exists elsewhere: 1 <-> 2
    both = ring + [(1, 2), (2, 1)]
    assert girth(3, both) == 2
    assert girth(3, [(0, 1), (1, 2)]) == -1
    # weighted: the cheap way back is 0 -> 1 -> 0 (cost 3)
    weighted = [(0, 1, 1), (1, 0, 2),
                (0, 2, 1), (2, 0, 9)]
    assert weighted_cycle_through(3, weighted, 0) == 3
```

**NOTES FROM CANDIDATES**

Strictly directed; do not relax to undirected.
BFS layer 0 is v itself; the first re-discovery during expansion is the cycle length.
Common bug: marking v as visited immediately blocks the re-entry.

**PREPARATION**

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).